

Section-A

1. Star and Number Patterns

Time Complexity: $O(N^2)$

Space Complexity: $O(1)$

Explanation - The pattern uses nested loops. The total no. of rows and elements printed is proportional to N^2 . Only loop variables are used, so constant extra space is required.

2. Linear Search

Time Complexity: $O(N)$

Space Complexity: $O(1)$

Explanation: The algorithm checks elements one by one until target is found.

In the worst case, all N elements are checked.

Optimised Approach: Binary Search can be used on sorted data with $O(\log N)$ time.

3. Binary Search

Time Complexity: $O(\log N)$

Space Complexity: $O(1)$

The search space is divided into half after every comparison. Therefore, only $\log N$ comparisons are required.

4. Bubble Sort

Time Complexity: $O(N^2)$

Space Complexity: $O(1)$

~~The search space is divided.~~

The algorithm repeatedly compares adjacent elements using nested loops.

Optimized Approach: An optimized version with a swapped flag can achieve $O(N)$ in best case.

5. Insertion Sort

Time Complexity: $O(N^2)$

Space Complexity: $O(1)$

Each element is inserted into its correct position in the sorted part of the array.

Optimized Approach: For a large data, Merge Sort can be used.

6. Selection Sort

Time Complexity: $O(N^2)$

Space Complexity: $O(1)$

The algorithm repeatedly searches for the minimum element in the unsorted part of array.

Optimised Approach: Merge Sort can be used for large datasets with $O(N \log N)$ time.

7. Merge Sort

Time Complexity: $O(N \log N)$ Space Complexity: $O(N)$

The array is divided into halves recursively and then merged. Each level process N elements, and there are $\log N$ levels.

8. Quick Sort

Time Complexity: Average $O(N \log N)$, Worst: $O(N^2)$ Space Complexity: Average: $O(\log N)$, Worst: $O(N)$

The array is partitioned around a pivot and recursively sorted. Performance depends on choice of pivot.

9. Heap Sort

Time Complexity: $O(N \log N)$ Space Complexity: $O(1)$

A heap is built and max. element is repeatedly removed and placed in its correct position.

Section -B

Code

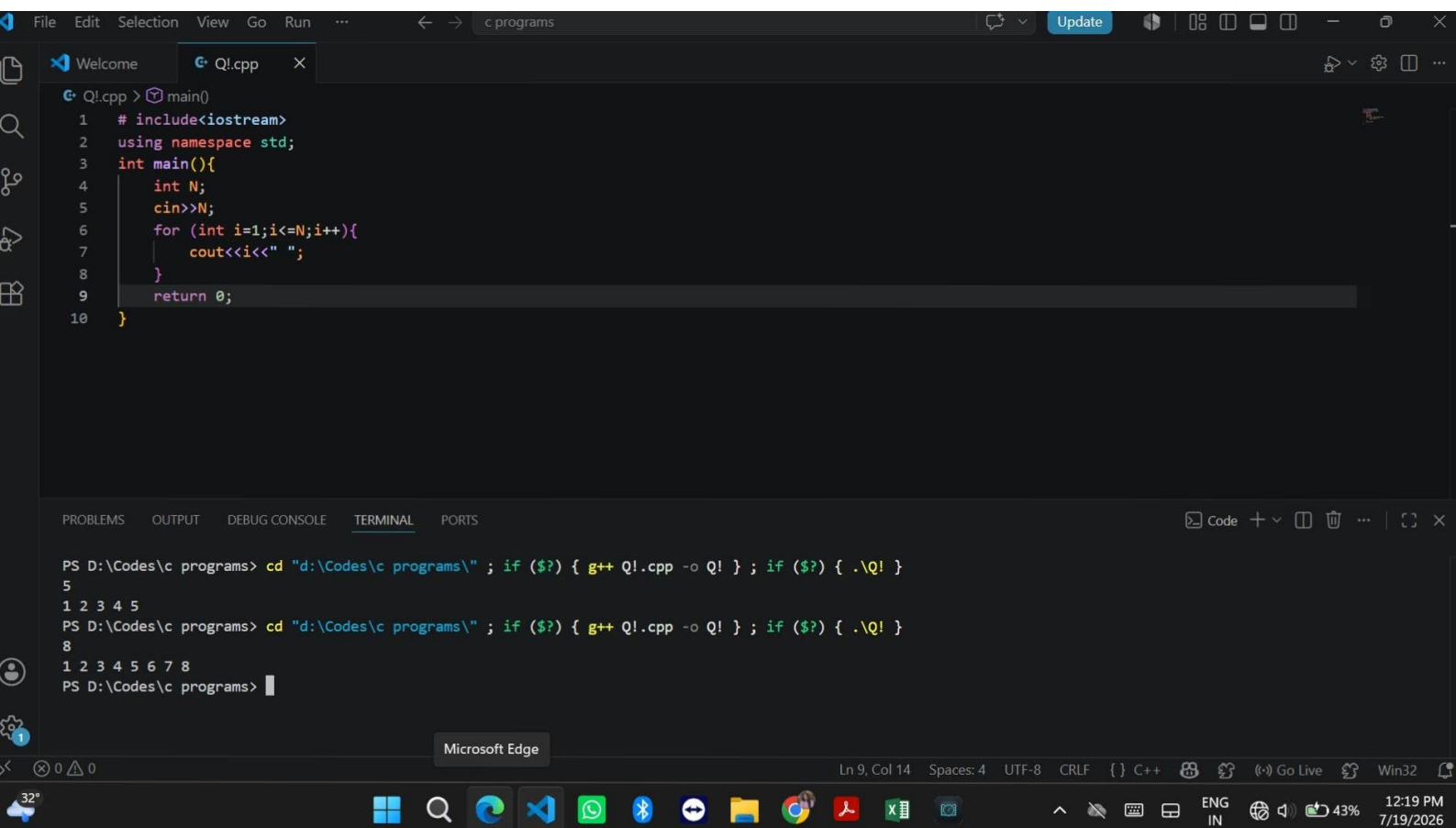
```
#include <iostream>
using namespace std;
int main() {
    int N;
    cin >> N;
    for (int i = 1; i <= N; i++) {
        cout << i << " ";
    }
    return 0;
}
```

3

Time Complexity
 $O(N)$

Space Complexity
 $O(1)$

The for loop runs exactly N times and each iteration prints one number. Therefore, the time complexity is $O(N)$. Only the variable i and N are used, so the algo. requires constant extra space, giving $O(1)$ space complexity.



②

```
#include <iostream>
using namespace std;
int main() {
    int N;
    int sum = 0;
    cin >> N;
    for (int i = 1; i <= N; i++)
    {
        sum = sum + i;
    }
    cout << sum;
    return 0;
}
```

Time Complexity
 $O(N)$

Space Complexity
 $O(1)$

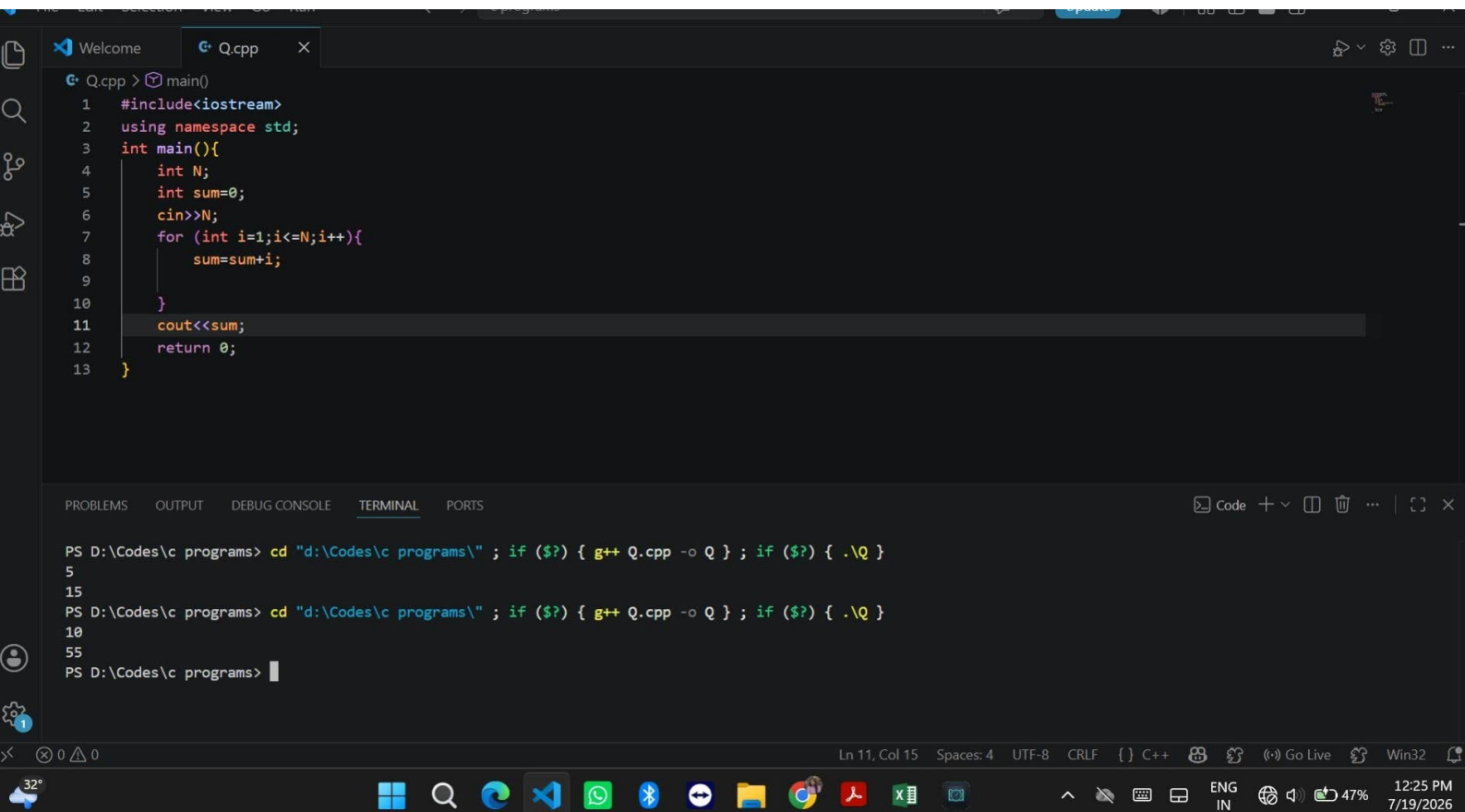
The loop runs from 1 to N, so it executes N times. Therefore time complexity is $O(N)$.

Only N, sum, i are used; so extra space required is constant; $O(1)$.

More Efficient Approach

$$\text{sum} = N * (N + 1) / 2;$$

This approach takes $O(1)$ time and $O(1)$ space.



③ #include <iostream>

using namespace std;

int main () {

int A, B;

int ans = 1;

cin << A;

cin << B;

for (int i = 1; i <= B; i++) {

ans = ans * A;

}

cout << ans;

return 0;

}

Time Complexity

$O(B)$

Space Complexity

$O(1)$

The loop executes B times and each iteration multiplies the current result by A. Therefore ; time complexity is $O(B)$.

Only (A, B, ans and i) are used so extra space complexity is $O(1)$.

Efficient Approach

Binary exponentiation can calculate A^B in $O(\log B)$ time by repeatedly squaring the base and reducing the exponent by half. It is more efficient than multiplying A by itself B times.

